Shortest Paths and Breadth-First Search

Design and Analysis of Algorithms

Brian C. Dean

School of Computing

Clemson University

Depth-First Search (DFS)

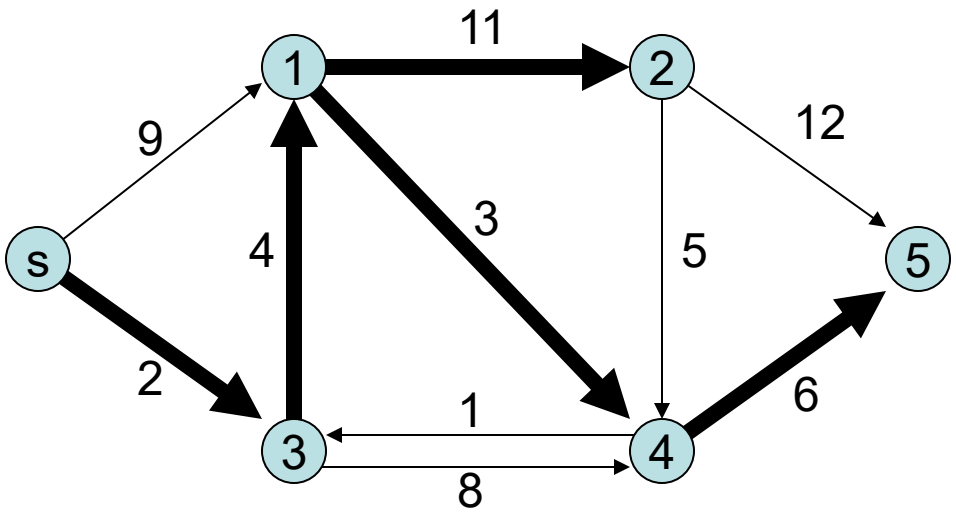
```
DFS-Visit(i):
Status[i] = 'visited'
For all j such that (i,j) is an edge:
    If Status[j] = 'unvisited':
        Pred[j] = i
        DFS-Visit(j)
```

- To find a path from i to j, launch DFS at i, then walk backward along pred[] pointers from j.
- Collectively, pred[] pointers form a directed tree rooted at i.
- Paths recovered in the same way in most shortest path algorithms, and also in dynamic programming.
- Let's see an example in code…

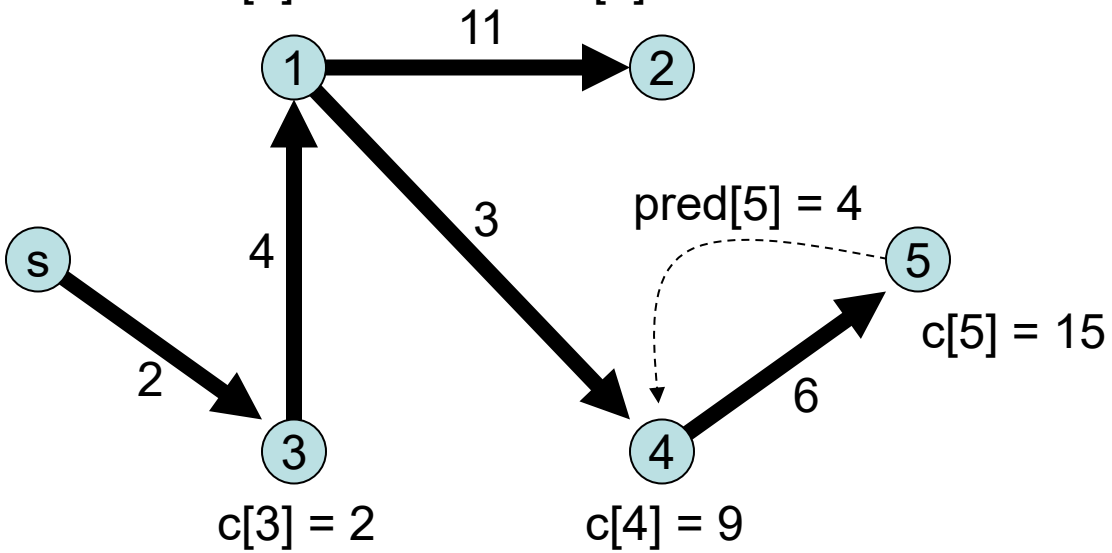
Types of Shortest Path Problems

- **Single-source single-destination:** find the shortest path from s to t in a (directed) graph.
- **Single-source:** find the shortest path from s to every other node in a (directed) graph.
- **All-pairs:** find the shortest path from every node to every other node.

Single-Source Shortest Paths



c[1] = 6 c[2] = 17



Input:

Directed graph with costs/weights/lengths on its edges.

Output:

- c[j]: cost of shortest $s \rightarrow j$ path
- pred[j]: previous node (before j) on a shortest $s \rightarrow j$ path.

Single-Source Shortest Paths: Algorithms

- Easy cases that we can solve in linear time:
  - Unweighted graphs: use **breadth-first search**.
  - Directed acyclic graphs (DAGs): use **dynamic programming**.
- If edge costs are nonnegative, we'll use **Dijkstra's algorithm**.
- If some edge costs are negative, we use the **Bellman-Ford algorithm**.
- If there is a negative cycle, we are out of luck!

Breadth-First Search

- Depth-first search dives as deeply as possible into a graph until it can go no further, then it backs up and tries to branch.
- In contrast, a **breadth-first search** (BFS) starting at some source node s will visit s, then all nodes 1 hop away from s, then all nodes 2 hops away from s, etc.
- Like DFS, we can also use BFS to find connected components or answer “find a path from i to j” queries.
  - BFS finds a path from i to j having fewest edges.

Breadth-First Search

```
BFS(s):
   For all nodes i:
      pred[i] = null
      dist[i] = Infinity
   Q = {s}
   dist[s] = 0
   While Q is nonempty:
       i = next node in Q
       For all nodes j such that (i,j) is an edge:
          If dist[j] = Infinity,
             pred[j] = i
             dist[j] = dist[i] + 1
             Append j to the end of Q
```

- If Q is implemented as a stack rather than as a FIFO queue, we get DFS rather than BFS!